

Criação do contexto técnico do projeto PixelPress (.ai/) —

atividade-2

Você está operando em **Agent Mode**, com permissão para criar arquivos e escrever documentação.

Atue como um **Staff Engineer / Tech Lead Node.js**, responsável por preparar o **contexto técnico e arquitetural** do **PixelPress**, que será implementado posteriormente por outro agente.

Pense como se estivesse escrevendo documentação interna para um time real.

Contexto do projeto (leia antes de tudo)

O **PixelPress** é um app web de descoberta e tracking de video games (estilo Letterboxd para jogos), já arquitetado nos documentos `01-funcionalidades-principais.md` a `07-fluxos-principais.md`, na raiz do repositório. **Leia esses sete arquivos antes de escrever qualquer linha**; eles são a fonte de verdade do domínio (funcionalidades, papéis RBAC, entidades, endpoints, fluxos).

Resumo do domínio (não substitui a leitura dos docs):

- Catálogo de jogos proveniente da **RAWG Video Games Database API** (read-only, integração HTTP real).
- Backend próprio para dados do usuário: biblioteca pessoal com status, reviews, listas customizadas, follow social, wishlist, perfil com estatísticas.
- Autorização **RBAC** com 4 papéis: Visitante, Usuário registrado, Moderador, Administrador.
- Entidades (item 4): `Usuario`, `Papel`, `Jogo` (referência ao catálogo RAWG), `ItemBiblioteca`, `Review`, `Lista`, `ItemLista`, `Follow`, `ItemWishlist`.
- API REST versionada sob `/api/v1` (item 5).

Decisão de escopo da atividade-2

Esta atividade é a fase de **implementação do backend**. A única restrição em relação aos docs 1–7:

1. **Backend REST apenas**. Sem frontend, sem SPA.

A stack completa dos itens 3 e 6 se aplica integralmente: **NestJS + Prisma + PostgreSQL + Redis + RAWG API real**.

Objetivo

Criar uma pasta `.ai/` na raiz do projeto contendo documentos **claros, prescritivos e não genéricos**, que definem:

- padrões de código
- decisões arquiteturais
- stack tecnológica aprovada
- regras de negócio e domínio do PixelPress

Esses arquivos **não são explicativos**; eles **definem regras** que devem ser seguidas pelo agente implementador.

Estrutura obrigatória

Criar exatamente a seguinte estrutura:

```
.ai/  
├─ standards.md  
├─ architecture.md  
├─ tech-stack.md  
└─ business-rules.md
```

Conteúdo esperado (DETALHADO)

♦ `.ai/standards.md` — Padrões de código e estilo

Defina regras **concretas**, incluindo:

- **Node.js LTS** (mínimo v20); **TypeScript** com `strict: true` obrigatório.
- Uso de:
 - `class` para DTOs com decorators de `class-validator`. Nunca interfaces soltas como tipo de parâmetro de controller.
 - `readonly` em propriedades de DTOs de response.
 - `async/await` em toda operação de I/O (banco, cache, RAWG). Nunca `.then().catch()` inline.
- Convenções NestJS:
 - Controllers finos: recebem request, delegam ao service, devolvem response. Zero lógica de negócio.
 - Lógica de negócio **somente em Services**, incluindo validações de domínio e interação com Prisma e RawgClient.
 - Autorização (RBAC + ownership) em **Guards** NestJS, **nunca no controller**.
- Organização de módulos NestJS (um módulo por domínio):

```
src/  
  auth/  
  usuarios/  
  jogos/           ← integração RAWG + referência local via Prisma  
  biblioteca/  
  reviews/  
  listas/  
  social/          ← follow + feed  
  wishlist/  
  moderacao/  
  common/          ← guards, filters, pipes, interceptors, decorators globais  
  config/  
  prisma/  
    schema.prisma   ← fonte de verdade do schema  
    migrations/  
    seed.ts
```

- Nomenclatura de domínio **em português**, consistente com o item 4: `Usuario`, `Jogo`, `ItemBiblioteca`, `Review`, `Lista`, `ItemLista`, `Follow`, `ItemWishlist`.
- `PrismaService` é o único ponto de acesso ao banco; nunca instanciar `PrismaClient` diretamente fora dele.
- Status de biblioteca como **enum** (TypeScript + Prisma): `JOGANDO`, `ZERADO`, `QUERO_JOGAR`, `DROPEI`, `PLATINADO`.
- Versionamento da API sob `/api/v1`.

- Paginação padronizada: query params `page` e `page_size` (default 20, máx 100); resposta com `count`, `next`, `previous`, `results`. Implementar via `skip / take` do Prisma.
- Envelope de erro único via `ExceptionHandler` global:

```
{ "error": { "code": "...", "message": "...", "details": [] } }
```

- `ValidationPipe` global com `whitelist: true`, `forbidNonWhitelisted: true` e `transform: true`.
- Logger estruturado com `nestjs-pino`. Nenhum `console.log` em código de produção.
- Código legível > abstrações complexas.

Evite qualquer linguagem vaga como "quando possível".

♦ `.ai/architecture.md` — Arquitetura e decisões

Descreva explicitamente:

- Arquitetura em **módulos NestJS**: `Controller` → `Service` → `Repository` (Prisma) → `PostgreSQL`.
- Catálogo de jogos via **RAWG API real** (HTTP), encapsulado no módulo `jogos/` via `RawgClient` (`@nestjs/axios`). Nenhum outro módulo chama a RAWG diretamente.
- Cache **Redis** (ioredis) usado exclusivamente para cache-aside das respostas da RAWG com TTL configurável. Dados do usuário nunca passam pelo Redis.
- Padrão cache-aside no fluxo do catálogo (item 7, Fluxo 2): consulta Redis → hit responde; miss chama RAWG, grava no Redis, responde.
- Entidade `Jogo` no banco armazena referência mínima (`rawgId`, `slug`, `nome`, `capaUrl`); criada via upsert no primeiro uso. Detalhes ricos buscados sob demanda da RAWG, nunca persistidos.
- Seed via `prisma/seed.ts`: pelo menos 4 usuários (um por papel), alguns `ItemBiblioteca`, `Review`, `Lista` de exemplo e 1 denúncia pendente. Jogos do seed referenciam `rawgId`s reais.
- Autenticação **stateless via JWT** (`@nestjs/passport` + `passport-jwt`). Autorização **RBAC** via `RolesGuard` + `@Roles(...)`. Ownership validado no service (403).
- Hierarquia de papéis: `ADMIN` ≥ `MODERADOR` ≥ `USUARIO`.
- Em falha da RAWG: retornar **502 Bad Gateway**. Chave RAWG nunca exposta em logs ou respostas.
- Infraestrutura local via **Docker Compose**: serviços `api`, `postgres`, `redis`.

Inclua **3 decisões arquiteturais** no formato:

```
Decision:
Context:
Consequence:
```

Sugestões: (1) Prisma vs. TypeORM; (2) Redis cache-aside vs. cache in-memory; (3) JWT stateless vs. sessões.

♦ `.ai/tech-stack.md` — Stack tecnológica (PRESCRITIVO)

Defina explicitamente:

- **Node.js LTS** (mínimo v20).
- **NestJS** (mínimo v10), **TypeScript** com `strict: true`.
- Gerenciador de pacotes: **pnpm** (preferido) ou npm.
- Dependências **aprovas** (runtime):
 - `@nestjs/core`, `@nestjs/common`, `@nestjs/platform-express`
 - `@nestjs/passport`, `passport`, `passport-jwt`, `@nestjs/jwt`

- o `bcrypt`
- o `@nestjs/swagger`, `swagger-ui-express`
- o `class-validator`, `class-transformer`
- o `@nestjs/config`
- o `@nestjs/axios`, `axios` (cliente HTTP para RAWG)
- o `@prisma/client` (ORM, acesso ao PostgreSQL)
- o `ioredis` (cache Redis)
- o `pino`, `nestjs-pino` (logging estruturado)
- o `reflect-metadata`, `rxjs`
- Dependências **aprovadas** (devDependencies):
 - o `prisma` (CLI migrations/seed), `ts-node`
 - o `@nestjs/testing`, `jest`, `ts-jest`, `supertest`
 - o Types: `@types/node`, `@types/passport-jwt`, `@types/bcrypt`, `@types/express`, `@types/jest`, `@types/supertest`
- Dependências **proibidas**:
 - o `node-fetch`, `got`, `undici` (usar `@nestjs/axios`)
 - o `typeorm`, `sequelize`, `mikro-orm` (Prisma é o ORM aprovado)
 - o `mongoose` (banco é relacional)
 - o `sqlite3`, `better-sqlite3` (banco de produção é PostgreSQL)
 - o `jsdom`, `cheerio`, `puppeteer`, `playwright` (não é crawler)
- Configuração via `@nestjs/config` com arquivo `.env`. Variáveis obrigatórias:

```
DATABASE_URL, REDIS_URL, JWT_SECRET, JWT_EXPIRATION, JWT_REFRESH_EXPIRATION,
RAWG_API_KEY, RAWG_BASE_URL, RAWG_CACHE_TTL_SECONDS
```

- `.env` nunca versionado; `.env.example` com placeholders deve ser versionado.
- Infraestrutura local: **Docker Compose** com `api`, `postgres`, `redis`.

Declare claramente:

Qualquer dependência fora desta lista não deve ser utilizada sem decisão arquitetural documentada.

♦ `.ai/business-rules.md` — Regras de negócio

Documente regras claras, derivadas dos itens 1–7. Não inclua código.

Catálogo (RAWG API real):

- Catálogo fornecido pela RAWG API real via HTTP. Nenhum dado de jogo é fictício ou fabricado.
- **Jogo** é referência mínima no banco (`rawgId`, `slug`, `nome`, `capaUrl`); detalhes ricos buscados sob demanda.
- Catálogo é read-only para todos os papéis. Nenhum endpoint permite criar, editar ou deletar jogos.
- Respostas da RAWG cacheadas no Redis com TTL configurável.
- Chave RAWG configurada via `.env`; nunca exposta em logs ou respostas. Em falha da RAWG: 502.

Autorização (RBAC, item 2):

- 4 papéis hierárquicos/cumulativos: **Visitante**, **Usuário registrado**, **Moderador** ($\supseteq$ Usuário), **Administrador** ($\supseteq$ Moderador).
- Reproduza a **matriz de permissões** do item 2 (recurso/ação × papel), com ,  e `próprio`.
- **Ownership**: usuário só edita/exclui biblioteca, reviews, listas e wishlist próprios; caso contrário, 403.
- **Visibilidade**: listas `publica: false` visíveis apenas ao dono e a Admin; Visitante recebe 404.
- **Atribuição de papéis**: apenas Admin concede/revoga; ninguém eleva o próprio papel.

Regras de domínio (itens 4 e 7):

- Status de biblioteca restrito ao enum: `JOGANDO`, `ZERADO`, `QUERO_JOGAR`, `DROPEI`, `PLATINADO`. Violação: 422.
- Unicidade reforçada por `@unique` no schema Prisma e validada no service antes da escrita:
 - `ItemBiblioteca(usuarioId, jogoId)`, `Review(usuarioId, jogoId)`, `ItemWishlist(usuarioId, jogoId)`, `Follow(seguidorId, seguidoId)`. Violação: 409.
- Nota de review: intervalo 0–10 (inteiro); fora disso, 422.
- **Soft-delete de moderação:** Moderador seta `oculto: true` + registra `motivoOcultacao` e `ocultadoPorId`. Nunca apaga fisicamente.
- Fluxo de denúncia: denúncia (status `PENDENTE`) → Moderador lista → Moderador oculta (status `RESOLVIDA`).

Convenções de API:

- Sem token → 401. Sem papel/ownership → 403. Não encontrado → 404. Conflito de unicidade → 409. Regra violada → 422. Falha na RAWG → 502.
- Endpoints versionados sob `/api/v1`. Catálogo read-only (escrita retorna 405).
- Toda resposta de erro segue o envelope `{ "error": { "code", "message", "details" } }`.

Regras finais

- Escreva os documentos pensando em **outro agente** (o implementador), não em um leitor humano casual.
- Seja específico e direto; ancore tudo nos itens 1–7.
- Não use frases genéricas.
- Não antecipe implementação (não escreva código de produção dentro dos `.ai/`).

Entregável

- Pasta `.ai/` criada na raiz.
- Quatro arquivos preenchidos conforme especificado.
- Ao final, apresente:
 - Lista dos arquivos criados.
 - Um resumo objetivo de cada documento.
- Em seguida, **aguarde mais instruções**.

Implementação do MVP backend do PixelPress — atividade-2

Você está operando em **Agent Mode**, com permissão para criar diretórios, escrever arquivos, instalar dependências e executar comandos.

Atue como um **engenheiro de software sênior Node.js/NestJS**, implementando o backend do **PixelPress** a partir do contexto técnico já definido na pasta `.ai/`.

Ferramenta sugerida: IDE/plugin de agentes do grupo (ex.: Antigravity, Cursor, Claude Code).

Contrato vinculante (leia antes de tudo)

A pasta `.ai/` na raiz do repositório é a **fonte de verdade** desta implementação. **Leia os quatro arquivos antes de escrever qualquer linha de código** e trate cada regra como obrigatória, **exceto** os desvios conscientes declarados na seção seguinte:

#	Arquivo	O que define
1	<code>.ai/standards.md</code>	Convenções de código, estrutura de módulos, DTOs, paginação, envelope de erro
2	<code>.ai/architecture.md</code>	Camadas, integração RAWG real, cache-aside, JWT, RBAC, seed
3	<code>.ai/tech-stack.md</code>	Versões, dependências aprovadas e proibidas , variáveis de ambiente
4	<code>.ai/business-rules.md</code>	RBAC, matriz de permissões, ownership, unicidade, fluxo de denúncia

Os sete documentos `01-...md` a `07-...md` na raiz são o domínio original e podem ser consultados como referência secundária. Em qualquer conflito **não listado abaixo**, `.ai/` **prevalece**.

Regras inegociáveis que **permanecem** valendo:

- **Catálogo via RAWG API real** (HTTP), encapsulado no módulo `jogos/` pelo `RawgClient` (`@nestjs/axios`). Nenhum outro módulo chama a RAWG diretamente.
- Chave RAWG (`RAWG_API_KEY`) via `.env` ; **nunca** exposta em logs, respostas ou código versionado. Em falha da RAWG (timeout, 4xx, 5xx): **502 Bad Gateway** com mensagem padronizada.
- `Jogo` no banco é referência mínima (`rawgId` , `slug` , `nome` , `capaUrl`), criada via upsert no primeiro uso. Detalhes ricos buscados sob demanda, nunca persistidos.
- TypeScript com `strict: true` , sem `any` explícito.
- Controllers finos; lógica de negócio só em services; RBAC e ownership em guards/services conforme `standards.md` .
- `PrismaService` é o único acesso ao banco; nunca instanciar `PrismaClient` fora dele.
- Nomenclatura de domínio **em português** (`Usuario` , `Jogo` , `ItemBiblioteca` , `Review` , `Lista` , `Follow` , `ItemWishlist`).
- Envelope de erro, paginação e códigos HTTP exatamente como em `standards.md` .
- Nenhuma dependência da lista **proibida** além da exceção declarada abaixo (`sqlite`). Continuam vetados `typeorm` , `sequelize` , `mongoose` , `node-fetch` , `got` , `undici` , `puppeteer` , `cheerio` , etc.

Desvios conscientes do `.ai/` (decisão de escopo do MVP)

Para esta entrega o grupo optou por uma demo de **baixa infraestrutura** (sem Docker, sem serviços externos além da própria RAWG). Dois pontos do `.ai/` são deliberadamente sobrescritos **apenas neste MVP**, e o agente deve registrá-los no relatório final:

#	Regra original no .ai/	Override no MVP	Justificativa
1	PostgreSQL como banco; <code>sqlite</code> proibido	SQLite em arquivo via Prisma (<code>provider = "sqlite"</code> , <code>DATABASE_URL="file:./dev.db"</code>), gitignored e recriado/seedado a cada boot	Demo roda com um único <code>pnpm start:dev</code> , sem Docker; arquivo local evita os problemas de connection-pool do <code>:memory:</code>
2	Cache-aside em Redis (<code>ioredis</code>)	Cache-aside in-process (serviço com <code>Map</code> + TTL, sem dependência nova)	Cumprir o papel do cache do <code>architecture.md</code> e protege o rate limit da RAWG, sem subir Redis

O catálogo **continua sendo a RAWG real** (isto **não** é desvio: é o que o `.ai/` sempre pediu). O Prisma **permanece** como ORM e único acesso ao banco. Só mudam o `provider` do banco e o backend do cache. **Nenhum outro desvio é permitido**: persistência real, JWT real, RBAC real, validações reais.

Consequência técnica obrigatória: o conector SQLite do Prisma **não suporta** `enum` . Modele `Papel` e `StatusBiblioteca` como `String` no `schema.prisma` , mantendo o `enum TypeScript` como fonte de verdade e validando com `@IsEnum(...)` nos DTOs.

Objetivo desta atividade

Entregar um **MVP backend executável localmente** que demonstre o domínio do PixelPress numa apresentação de 3 a 10 minutos, com **catálogo real da RAWG** e **dados de usuário mockados** (seed).

Materialização:

- Catálogo real.** O `RawgClient` consome a **RAWG API real** via `@nestjs/axios` , protegido por **cache-aside in-process** (TTL configurável) para respeitar o rate limit de 100 req/dia do plano gratuito.
- Fallback offline opcional.** Um `MockRawgClient` (fixture de jogos reais) fica disponível e é injetado quando `USE_RAWG MOCK=true` . Default é `false` (usa a API real). Serve como rede de segurança caso a internet ou a chave falhe **durante a apresentação ao vivo**.
- Banco SQLite em arquivo** (`file:./dev.db` , gitignored) via Prisma, recriado e seedado a cada boot.
- Dados de usuário mockados** via `prisma/seed.ts` (4 usuários, um por papel + um inativo, biblioteca, reviews e 1 denúncia pendente).
- Execução local** com um único comando após `pnpm install` e uma chave RAWG no `.env` . Sem Docker.

Escopo do MVP (vertical slice demonstrável)

Implemente um corte vertical coeso que exercite catálogo real, RBAC, ownership e o fluxo de moderação. Módulos **dentro** do MVP:

#	Módulo	Entrega mínima
1	<code>config/</code>	<code>ConfigModule</code> global validando as variáveis de <code>.env</code>
2	<code>common/</code>	<code>ValidationPipe</code> , <code>ExceptionHandler</code> global (envelope de erro), <code>RolesGuard</code> , <code>@Roles()</code> , decorator <code>@UsuarioAtual()</code> , helper de paginação, <code>CacheService</code> in-process
3	<code>auth/</code>	Registro, login (JWT access + refresh), <code>JwtStrategy</code> , hash com <code>bcrypt</code>
4	<code>usuarios/</code>	CRUD mínimo de perfil + atribuição de papel (apenas Admin)
5	<code>jogos/</code>	Busca e detalhe via <code>RawgClient</code> (RAWG real) + cache-aside in-process + upsert da referência <code>Jogo</code>

#	Módulo	Entrega mínima
6	biblioteca/	Adicionar/atualizar/remover <code>ItemBiblioteca</code> com status (enum TS), unicidade (<code>usuarioId</code> , <code>jogoId</code>), ownership
7	reviews/	Criar/editar/excluir review (nota 0–10, unicidade, ownership) + rota de moderação <code>hide</code>
8	moderacao/	Criar denúncia (<code>PENDENTE</code>), listar pendentes (Moderador), ocultar conteúdo (soft-delete <code>oculto: true</code>)

Módulos **fora** do MVP (deixe declarados mas vazios, ou omita, e registre no relatório): `listas/` , `social/` , `wishlist/` . Não os implemente agora; mantenha a estrutura prevista em `standards.md` sem quebrar o build.

Plano de execução (ordem obrigatória para o agente)

Execute em fases. Ao final de cada fase, garanta que o projeto **compila** antes de avançar.

Fase 0 — Bootstrap

1. Inicialize o projeto NestJS 10 com `pnpm` , `TypeScript strict` , estrutura de pastas exata de `standards.md` .
2. Crie `tsconfig.json` com `strict: true` , `target: ES2022` , `module: commonjs` .
3. Instale as dependências aprovadas em `tech-stack.md` , **com estas diferenças do MVP**:
 - o **Remova** `ioredis` (cache passa a ser in-process, sem dependência nova).
 - o **Mantenha** `@nestjs/axios` + `axios` (cliente da RAWG real).
 - o `prisma` / `@prisma/client` usam o provider **sqlite**.
4. Crie `.env.example` (versionado) e `.env` (não versionado). Variáveis do MVP:

```

DATABASE_URL="file:./dev.db"
JWT_SECRET=<string longa e aleatória>
JWT_EXPIRATION=3600s
JWT_REFRESH_EXPIRATION=7d
RAWG_API_KEY=<chave real obtida em rawg.io>
RAWG_BASE_URL=https://api.rawg.io/api
RAWG_CACHE_TTL_SECONDS=300
USE_RAWG MOCK=false

```

Não inclua `REDIS_URL` .

5. **Não** crie `docker-compose.yml` . A demo não usa Docker.

Fase 1 — Persistência (SQLite em arquivo)

6. Escreva `prisma/schema.prisma` com `provider = "sqlite"` : `models Usuario` , `Jogo` , `ItemBiblioteca` , `Review` , `Denuncia` , com as constraints `@unique` de `business-rules.md` (`ItemBiblioteca(usuarioId, jogoId)` , `Review(usuarioId, jogoId)`). Campos de papel/status como `String` (SQLite não tem `enum`).
7. Use o script `pnpm prisma db push` para materializar o schema no `dev.db` , e adicione um `pnpm db:reset` (apaga `dev.db` , refaz push e roda o seed). `dev.db` vai no `.gitignore` . Para a demo, o boot deve garantir banco populado: rode `db push` + seed antes de `start:dev` (ou no `prestart`).
8. Implemente `PrismaService` (provider global, conecta no `onModuleInit`).

Fase 2 — Plataforma transversal (`common/` + `config/`)

9. `main.ts` : `ValidationPipe` `global( whitelist , forbidNonWhitelisted , transform )`, `nestjs-pino` , prefixo `/api/v1` , `Swagger` em `/api/docs` .
10. `ExceptionHandler` `global` emitindo o envelope `{ "error": { "code", "message", "details" } }` com os códigos HTTP de `standards.md` .
11. `RolesGuard` + `@Roles()` lendo o papel do JWT; hierarquia `ADMIN ≥ MODERADOR ≥ USUARIO` .
12. Helper de paginação (`page` , `page_size` , envelope `count / next / previous / results`) via `skip / take` do `Prisma` .
13. `CacheService` in-process: chave → `{ valor, expiraEm }` num `Map` , com TTL de `RAWG_CACHE_TTL_SECONDS` . Métodos `get / set` assíncronos para manter o contrato `cache-aside` do `architecture.md` . **Exclusivo para respostas da RAWG**; dados de usuário nunca passam por ele.

Fase 3 — Catálogo via RAWG real (jogos/)

14. Defina a interface `RawgClient` (busca, detalhe) no módulo `jogos/` .
15. Implemente **duas** classes que satisfazem o contrato:
 - o `HttpRawgClient` — **caminho principal**. Usa `@nestjs/axios` , lê `RAWG_API_KEY` e `RAWG_BASE_URL` , normaliza o payload da RAWG. Em `timeout/4xx/5xx`, lança erro que o `ExceptionHandler` mapeia para **502**.
 - o `MockRawgClient` — **fallback offline**. Serve `jogos.fixture.json` (~12 jogos reais da RAWG: `rawgId` , `slug` , `nome` , `capaUrl` e alguns detalhes ricos).
16. No `JogosModule` , `factory provider` lê `USE_RAWG MOCK : false` → `HttpRawgClient` (default); `true` → `MockRawgClient` .
17. `JogosService` aplica **cache-aside** via `CacheService` sobre o resultado do client: consulta o cache → hit responde sem chamar a RAWG; miss chama a RAWG, grava no cache com TTL, responde. Faz **upsert** da referência mínima `Jogo` no banco no primeiro uso.

Fase 4 — Domínio do usuário

18. `auth/` : registro + login emitindo JWT; senha com `bcrypt` ; `JwtStrategy` populando `req.user` .
19. `biblioteca/` , `reviews/` , `moderacao/` conforme escopo, com validações de domínio **no service**, nunca no controller: `@IsEnum(StatusBiblioteca)` , nota 0–10 (422), unicidade (409), ownership (403).
20. Fluxo de denúncia completo de `business-rules.md` : criar (`PENDENTE`) → listar pendentes (Moderador) → ocultar (`oculto: true` , `motivoOcultacao` , `ocultadoPorId` , denúncia → `RESOLVIDA`).

Fase 5 — Seed mockado

21. `prisma/seed.ts` populando: 4 usuários (`USUARIO` , `MODERADOR` , `ADMIN` e um inativo), com senhas conhecidas para a demo; alguns `ItemBiblioteca` e `Review` referenciando `rawgId` s reais da RAWG; e **1 denúncia pendente** para demonstrar moderação. Documente as credenciais no `README` .

Fase 6 — Verificação e entrega

22. Garanta `pnpm build` sem erros e `pnpm start:dev` subindo a API com o banco já populado.
23. Execute o smoke test abaixo e cole a saída no relatório final.

✅ Definition of Done (critérios de aceite verificáveis)

O MVP está pronto quando, a partir de um clone limpo, esta sequência funciona **sem Docker** (com internet e uma chave RAWG válida):

```
cp .env.example .env          # preencha RAWG_API_KEY; DATABASE_URL="file:./dev.db" já default
pnpm install
```

```
pnpm db:reset # cria dev.db, aplica o schema e roda o seed
pnpm start:dev
```

E o smoke test passa:

#	Cenário	Comando / esperado
1	Health/docs	GET /api/docs carrega Swagger
2	Login seed	POST /api/v1/auth/login com credencial do seed → 200 + JWT
3	Catálogo real (miss → hit)	GET /api/v1/games?search=zelda → 200 da RAWG real; segunda chamada idêntica servida do cache in-process (sem nova chamada à RAWG)
4	Detalhe	GET /api/v1/games/{slug} → 200 com detalhes da RAWG
5	Falha RAWG	com RAWG_API_KEY inválida, chamada ao catálogo → 502 no envelope padrão (chave nunca aparece na resposta)
6	Biblioteca + ownership	usuário adiciona ItemBiblioteca ; outro usuário tentando editar → 403
7	Unicidade	adicionar o mesmo jogo 2× à biblioteca → 409
8	Review inválida	nota: 11 → 422
9	Sem token	escrita sem Authorization → 401
10	Fluxo de denúncia + RBAC	usuário denuncia review → Moderador lista pendentes → Moderador oculta → denúncia RESOLVIDA , review some da listagem pública; usuário comum tentando ocultar → 403

Todos os erros devem sair no envelope { "error": { "code", "message", "details" } } .

⚠ Regras finais para o agente

- Siga o `.ai/` em tudo, **exceto** os dois desvios declarados (SQLite em arquivo e cache in-process no lugar do Redis). Não introduza nenhum outro desvio para "facilitar".
- **O catálogo é a RAWG real.** O `MockRawgClient` existe só como fallback offline sob `USE_RAWG MOCK=true` ; default é a API real.
- Respeite o rate limit: toda leitura de catálogo passa pelo cache-aside antes de chamar a RAWG. Nunca chame a RAWG fora do `RawgClient` .
- Não exponha `RAWG_API_KEY` em logs, respostas ou commits. Falha da RAWG → 502 sem vaziar detalhes.
- Como o banco é efêmero, **toda** a massa de demo vem do seed no boot; não dependa de estado persistido entre execuções.
- Commits pequenos e descritivos por fase. Código legível acima de abstração prematura (`standards.md`).

📦 Entregável final (o agente deve produzir)

1. Backend NestJS rodando localmente conforme a Definition of Done (um `pnpm start:dev` + chave RAWG, sem Docker).
2. `README.md` com: passo a passo de execução, como obter a chave RAWG, credenciais do seed, nota sobre o `dev.db` SQLite recriado via `db:reset` , e como ligar o fallback `USE_RAWG MOCK` para demo offline.
3. Relatório final em texto contendo:
 - Lista dos arquivos e módulos criados (e os deixados fora do MVP).
 - **Os dois desvios conscientes do `.ai/`** (SQLite em arquivo, cache in-process) e seu impacto.

- Saída do smoke test (cenários 1–10).
- Outras decisões tomadas que não estavam explícitas no `.ai/`.
- Dificuldades encontradas com o agente escolhido (para os comentários da apresentação).

4. Em seguida, **aguarde mais instruções**.

Implementação do MVP frontend do PixelPress, atividade-2

(camada de apresentação)

Você está operando em **Agent Mode**, com permissão para criar diretórios, escrever arquivos, instalar dependências e executar comandos.

Atue como um **engenheiro de software sênior Frontend React/TypeScript**, implementando a SPA do **PixelPress** que consome o backend já entregue (`/api/v1`).

Ferramenta sugerida: IDE/plugin de agentes do grupo (ex.: Antigravity, Cursor, Claude Code).

Contrato vinculante (leia antes de tudo)

Este prompt complementa o `2-prompt-de-implementacao.md` (backend). O **backend já está implementado e rodando** na raiz do repositório (`src/` , Prisma SQLite, RAWG real/mock). O frontend **não reimplementa regra de negócio**: ele consome a API e reflete na UI as permissões e validações que o backend já impõe.

Fontes de verdade, na ordem de precedência:

#	Fonte	O que define
1	<code>.ai/architecture.md</code>	Camadas: a SPA é a camada de apresentação, fala só com a API REST via HTTPS/JSON+JWT, nunca acessa banco nem a RAWG diretamente
2	<code>.ai/business-rules.md</code>	RBAC (matriz de permissões), ownership, fluxo de denúncia, status de biblioteca, faixa de nota. A UI espelha isso (mostra/esconde ações por papel)
3	<code>1-Arquitetura/06-tecnologias.md</code>	Stack de frontend aprovada (ver desvio 1 abaixo)
4	Contrato HTTP vivo do backend	<code>/api/v1</code> em execução e o OpenAPI em <code>/api/docs</code> (JSON em <code>/api/docs-json</code>). É a especificação literal de rotas, payloads e envelope de erro

Regras inegociáveis:

- A SPA **só** conversa com `http://<host>/api/v1` . **Nunca** chama a RAWG diretamente, **nunca** acessa banco. A `RAWG_API_KEY` jamais aparece no frontend (nem em `.env` , nem em código, nem em request).
- TypeScript com `strict: true` , sem `any` explícito.** Use `unknown` + narrowing quando o tipo for incerto.
- Componentes finos e burros na borda; lógica de dados em hooks.** Nada de `fetch` solto dentro de componente de tela: toda I/O passa por uma camada de dados (hooks de query/mutation).
- O **envelope de erro do backend** (`{ "error": { "code", "message", "details" } }`) é tratado de forma central (interceptor + toast), nunca com `try/catch` espalhado por tela.
- O **contrato de paginação** (`count` , `next` , `previous` , `results`) é respeitado em todas as listas.
- RBAC na UI é cosmético, não é segurança.** Esconder/desabilitar botões por papel melhora UX, mas a autorização real é do backend. Nunca assuma que esconder o botão protege o recurso.
- Nomenclatura de domínio em português**, consistente com o backend (`Usuario` , `Jogo` , `ItemBiblioteca` , `Review` , `Denuncia` , `StatusBiblioteca` , `Papel`).
- Não introduza dependência fora da lista aprovada (seção Stack) sem decisão documentada.

⚠️ Desvios conscientes (registrar no relatório final)

#	Situação	Decisão	Justificativa
1	<code>tech-stack.md</code> é backend-only : não lista dependências de frontend aprovadas	A stack de frontend é puxada de <code>1-Arquitetura/06-tecnologias.md</code> (React + TS + Tailwind + shadcn/ui) e formalizada na seção Stack deste prompt	Evita o mesmo gap de rastreabilidade do backend; a fronteira fica declarada explicitamente
2	Armazenamento do JWT no browser	localStorage para access + refresh token, apenas nesta demo MVP	Cookie <code>httpOnly</code> seria o ideal de produção, mas exige ajuste de CORS/credenciais no backend já entregue. Para a demo de instância única, <code>localStorage</code> é aceitável e fica registrado como dívida
3	Layout do repositório	A SPA vive em frontend/ ao lado de <code>src/</code> (backend). pnpm, projetos independentes	Risco zero para o backend já testado. A arquitetura sugere monorepo <code>frontend / backend</code> ; mover o backend para <code>backend/</code> fica como evolução futura, fora deste MVP

Nenhum outro desvio é permitido. Sem mock de API no frontend: a demo consome o backend real (que por sua vez pode estar com `USE_RAWG_MOCK=true` para rodar offline).

🎯 Objetivo

Entregar uma **SPA executável localmente** que demonstre visualmente o domínio do PixelPress numa apresentação de 3 a 10 minutos, consumindo o backend real: catálogo da RAWG, autenticação JWT, biblioteca, reviews e o fluxo de moderação, com a UI refletindo papéis e ownership.

Materialização:

1. **Login com credenciais do seed** do backend (4 usuários, um por papel). Sessão via JWT, com refresh e logout.
2. **Catálogo real** consumido via `/api/v1/games` (busca paginada) e `/api/v1/games/{slug}` (detalhe rico).
3. **Biblioteca pessoal**: adicionar jogo (a partir do detalhe), trocar status, remover. Ownership refletido na UI.
4. **Reviews**: criar/editar/excluir a própria; listar as públicas de um jogo (as ocultas não aparecem).
5. **Moderação** (Moderador/Admin): listar denúncias pendentes, ocultar review. Denunciar review (qualquer usuário).
6. **RBAC visível**: a UI mostra ou esconde ações conforme o papel do usuário logado, fiel à matriz de `business-rules.md`.
7. **Execução local** com um único `pnpm dev`, apontando para o backend via `VITE_API_BASE_URL`.

📦 Escopo do MVP (vertical slice, espelha o backend)

#	Área	Entrega mínima
1	Infra/app shell	Vite + React + TS strict, roteamento, layout base (header com usuário/papel + logout), tema Tailwind/shadcn
2	Camada de dados	Cliente HTTP único (axios) com interceptor de <code>Authorization: Bearer</code> , tratamento central do envelope de erro, refresh em 401; hooks via TanStack Query
3	Tipos	Tipos do domínio derivados do OpenAPI do backend (<code>/api/docs-json</code>) ou declarados à mão espelhando os DTOs. Sem <code>any</code>
4	Auth	Tela de login (atalhos para as credenciais do seed), guarda de rota privada, persistência de sessão, logout

#	Área	Entrega mínima
5	Catálogo	Busca com paginação (<code>page / page_size</code>), grid de cards, tela de detalhe com capa, descrição, metacritic, gêneros, plataformas, screenshots
6	Biblioteca	Lista "minha biblioteca", adicionar a partir do detalhe (seleção de <code>StatusBiblioteca</code>), editar status/horas, remover. Tratar 409 (unicidade) e 403 (ownership) com mensagem clara
7	Reviews	Criar/editar/excluir a própria review (nota 0 a 10, spoiler, texto); listar reviews públicas por jogo. Surfacing do 422 (nota inválida) e 409 (review duplicada)
8	Moderação	Botão "denunciar" numa review (usuário registrado); painel de denúncias pendentes e ação "ocultar" (Moderador/Admin); review ocultada some da listagem pública
9	Admin	Tela mínima de usuários com atribuição de papel (<code>/usuarios/:id/papel</code>), visível só para Admin

Fora do escopo (espelha o backend, deixar declarado no relatório, não implementar): `listas` , `social` (follow/feed), `wishList` . Sem SSR, sem realtime, sem testes obrigatórios nesta fase.

Stack aprovada (frontend)

Runtime e build:

#	Pacote	Finalidade
1	<code>react</code> , <code>react-dom</code> (18.x)	Biblioteca de UI
2	<code>typescript</code> (5.x, <code>strict</code>)	Tipagem
3	<code>vite</code> + <code>@vitejs/plugin-react</code>	Dev server e build
4	<code>react-router-dom</code>	Roteamento SPA
5	<code>@tanstack/react-query</code>	Data fetching, cache, estados de loading/erro
6	<code>axios</code>	Cliente HTTP (interceptors de auth e erro)
7	<code>react-hook-form</code> + <code>zod</code> + <code>@hookform/resolvers</code>	Formulários e validação client-side (espelha as regras, não as substitui)
8	<code>tailwindcss</code> , <code>postcss</code> , <code>autoprefixer</code>	Estilização
9	<code>shadcn/ui</code> (Radix + utilitários)	Componentes de UI (button, input, card, dialog, table, toast, select, badge)

Dev/codegen (opcional, recomendado):

#	Pacote	Finalidade
1	<code>openapi-typescript</code>	Gerar tipos a partir de <code>/api/docs.json</code> do backend, garantindo ponta a ponta

Gerenciador: **pnpm**. Proibido: qualquer cliente HTTP alternativo (`fetch` solto na borda da tela é desencorajado; centralize no `axios`), libs de estado pesadas desnecessárias (Redux/MobX) para este escopo, e qualquer chamada direta à RAWG.

Contrato de integração com o backend

Base URL via env: `VITE_API_BASE_URL` (default `http://localhost:3000/api/v1`).

Atenção: no ambiente de demo o backend pode subir em outra porta (ex.: `3100`) se a `3000` estiver ocupada. O frontend deve ler a base da env, nunca hardcodar a porta.

Rotas que a SPA consome (todas já existem no backend):

#	Fluxo	Endpoint
1	Login / refresh	<code>POST /auth/login</code> , <code>POST /auth/refresh</code>
2	Catálogo	<code>GET /games?search=&page=&page_size=</code> , <code>GET /games/{slug}</code>
3	Perfil próprio	<code>GET /usuarios/me</code>
4	Admin usuários	<code>GET /usuarios</code> , <code>PATCH /usuarios/{id}/papel</code>
5	Biblioteca	<code>POST /biblioteca</code> , <code>GET /biblioteca/me</code> , <code>PATCH /biblioteca/{id}</code> , <code>DELETE /biblioteca/{id}</code>
6	Reviews	<code>GET /reviews?jogo={slug}</code> , <code>POST /reviews</code> , <code>PATCH /reviews/{id}</code> , <code>DELETE /reviews/{id}</code>
7	Moderação	<code>POST /reports</code> , <code>GET /moderation/reports?status=PENDENTE</code> , <code>PATCH /reviews/{id}/hide</code>

Convenções a respeitar no cliente:

- Anexar `Authorization: Bearer <access_token>` em toda chamada autenticada.
- Em `401`, tentar `POST /auth/refresh` uma vez; se falhar, limpar sessão e redirecionar ao login.
- Parsear sempre o envelope `{ error: { code, message, details } }` e exibir `message` (e `details` quando houver) num toast. Mapear `code` para mensagens amigáveis quando fizer sentido (ex.: `REVIEW_DUPLICADA`, `ITEM_BIBLIOTECA_DUPLICADO`, `NOTA_INVALIDA`, `OWNERSHIP_NEGADO`).
- Listas seguem `{ count, next, previous, results }`; a paginação da UI usa esses campos.
- `StatusBiblioteca` (`JOGANDO`, `ZERADO`, `QUERO_JOGAR`, `DROPEI`, `PLATINADO`) e `Papel` (`USUARIO`, `MODERADOR`, `ADMIN`) como enums/uniões TS espelhando o backend.

Plano de execução (ordem obrigatória para o agente)

Execute em fases. Ao final de cada fase, garanta que o projeto **compila** (`pnpm build` ou `tsc --noEmit`) antes de avançar.

Fase 0, Bootstrap

1. Criar `frontend/` com Vite (`react-ts`), `pnpm`, `tsconfig` com `strict: true`. Configurar Tailwind e inicializar `shadcn/ui` (button, input, card, dialog, table, toast, select, badge).
2. Criar `.env.example` e `.env` com `VITE_API_BASE_URL`. Não versionar `.env`.
3. Configurar React Router e TanStack Query (QueryClientProvider). App shell com header (nome + papel do usuário + logout) e área de conteúdo.

Fase 1, Camada de dados e tipos

4. (Recomendado) Com o backend rodando, gerar tipos via `openapi-typescript` a partir de `/api/docs.json`. Alternativa: declarar tipos do domínio à mão.
5. Cliente axios único (`api/http.ts`) com `baseURL` da env, interceptor de Bearer, interceptor de resposta que normaliza o envelope de erro e dispara refresh em 401.

6. Hooks de query/mutation por recurso (`useLogin` , `useGames` , `useGameDetail` , `useMinhaBiblioteca` , `useAddBiblioteca` , `useReviews` , `useCreateReview` , `useReports` , `useHideReview` , etc).

Fase 2, Auth

7. Contexto de sessão (usuário atual + tokens). Tela de login com atalhos das 4 credenciais do seed. Guarda de rota privada. Logout limpando sessão.

Fase 3, Catálogo

8. Tela de busca: input de `search` , grid de cards (capa, nome), paginação. Tela de detalhe por `slug` com os campos ricos da RAWG.

Fase 4, Biblioteca e Reviews

9. Na tela de detalhe: ação "adicionar à biblioteca" (com seletor de status) e bloco de reviews do jogo (listar públicas; criar/editar/excluir a própria). Tela "minha biblioteca" com editar status/horas e remover. Tratar 409, 403, 422 com mensagens claras.

Fase 5, Moderação e Admin

10. Ação "denunciar" numa review (usuário registrado). Painel de denúncias pendentes e botão "ocultar" (visível a Moderador/Admin). Tela de usuários com atribuição de papel (Admin). Após ocultar, a review some da listagem pública.

Fase 6, Polimento e demo

11. Estados de loading/erro/vazio em todas as listas. Badges de papel e de status. Garantir que a UI esconde ações sem permissão conforme a matriz de `business-rules.md` .
12. `pnpm build` sem erros e `pnpm dev` servindo a SPA conectada ao backend.

✔ Definition of Done (espelha o smoke do backend, na UI)

A partir de um clone limpo, com o **backend rodando** (`pnpm start:dev` na raiz) e a SPA apontando para ele:

```
cd frontend
cp .env.example .env      # ajuste VITE_API_BASE_URL para a porta do backend
pnpm install
pnpm dev
```

E a UI cumpre:

#	Cenário	Esperado na tela
1	Login seed	Logar com <code>usuario@pixelpress.dev</code> / <code>Senha@123</code> entra na app; header mostra nome e papel
2	Login inativo	<code>inativo@pixelpress.dev</code> falha com mensagem de credencial inválida (backend retorna 401)
3	Catálogo	Buscar "witcher" lista resultados reais; abrir o card mostra o detalhe rico
4	Biblioteca	Adicionar um jogo à biblioteca aparece em "minha biblioteca" com o status escolhido
5	Unicidade	Tentar adicionar o mesmo jogo de novo mostra mensagem de duplicado (409)

#	Cenário	Esperado na tela
6	Review inválida	Enviar review com nota 11 mostra erro de nota inválida (422), sem quebrar a tela
7	Ownership	Logado como outro usuário, não há ação de editar review/biblioteca de terceiros (e a API barra com 403 se forçado)
8	Sessão	Recarregar a página mantém a sessão; logout volta ao login; rota privada sem sessão redireciona
9	RBAC visível	Como <code>usuario</code> , o painel de moderação/admin não aparece; como <code>moderador</code> , aparece denúncias pendentes
10	Fluxo de moderação	<code>moderador</code> lista denúncias pendentes, oculta a review denunciada, e ela some da listagem pública do jogo

Entregável final (o agente deve produzir)

- SPA React rodando localmente conforme a Definition of Done (um `pnpm dev` + backend ativo).
- `frontend/README.md` com: passo a passo de execução, variável `VITE_API_BASE_URL`, nota sobre a porta do backend, credenciais do seed e como rodar com o backend em modo `USE_RAWG MOCK=true` para demo offline.
- Relatório final em texto contendo:
 - Lista das telas, hooks e componentes criados (e o que ficou fora do MVP: `listas`, `social`, `wishlist`).
 - Os **desvios conscientes** (stack via `06-tecnologias.md`, JWT em `localStorage`, layout `frontend/`) e seu impacto.
 - Resultado dos 10 cenários da Definition of Done.
 - Decisões não explícitas nas fontes (ex.: geração de tipos via OpenAPI, mapa de `code` de erro para mensagens).
 - Dificuldades com o agente escolhido (para os comentários da apresentação).
- Em seguida, **aguarde mais instruções**.

Regras finais para o agente

- A SPA é camada de apresentação: **não** reimplementa regra de negócio, **não** chama a RAWG, **não** acessa banco. Tudo via `/api/v1`.
- A `RAWG_API_KEY` nunca entra no frontend. Nenhum segredo do backend no bundle.
- RBAC na UI é UX, não segurança: o backend continua sendo a autoridade. Não confie no esconder-botão.
- Não toque no código do backend (`src/`, `prisma/`). Este prompt é aditivo, isolado em `frontend/`.
- Commits pequenos e descritivos por fase. Legibilidade acima de abstração prematura.